

NOIP2026 模拟赛

By Cap1taL

时间：2026 年 8 月 1 日 08:30 ~ 13:00

题目名称	Anon 的花园	Soyo 的定向	Tomori 的石头	Taki 的融合
题目类型	传统型	传统型	传统型	传统型
目录	anon	soyo	tomori	taki
可执行文件名	anon	soyo	tomori	taki
输入文件名	anon.in	soyo.in	tomori.in	taki.in
输出文件名	anon.out	soyo.out	tomori.out	taki.out
每个测试点时限	3.0 秒	1.0 秒	1.0 秒	2.0 秒
内存限制	512 MiB	512 MiB	512 MiB	1024 MiB
测试点数目	10	10	20	20
测试点是否等分	是	是	是	是

提交源程序文件名

对于 C++ 语言	anon.cpp	soyo.cpp	tomori.cpp	taki.cpp
-----------	----------	----------	------------	----------

编译选项

对于 C++ 语言	- O2 - std=c++14 - static
-----------	---------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. `main` 函数的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 需要建立子文件夹。
4. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
5. 选手提交的程序源文件必须不大于 100KB。
6. 程序可使用的栈空间内存限制与题目的内存限制一致。
7. 四个题目的文件名没有任何特殊含义。

Anon 的花园 (anon)

【题目描述】

Chihaya Anon 在打扫她的花园.....
花园是一个一个 $n \times n$ 的矩阵，矩阵每格种着颜色为 $a_{i,j}$ 的草甸。
给定 Anon 的讨厌值 k ，Anon 想让你帮忙对矩阵内每一格草甸，求出以该格作为左上角且不越界的最大正方形边长，要求正方形区域内部出现的不同颜色草甸总数不超过 k 。
对每个位置输出对应的最大合法正方形边长 len 。

【输入格式】

第一行两个整数 n, k 。
接下来 n 行，每行 n 个整数，代表 $n \times n$ 的草甸颜色矩阵。

【输出格式】

输出共 n 行，每行 n 个整数，依次对应花园中每个格子作为左上角位置对应的最大合法正方形边长。

【样例 1】

输入

1	5	4			
2	1	2	3	3	2
3	4	5	1	3	1
4	2	3	5	5	5
5	5	3	1	2	3
6	3	3	3	1	2

输出

1	2	4	3	2	1
2	2	4	3	2	1
3	3	3	3	2	1
4	2	2	2	2	1
5	1	1	1	1	1

【样例 2】

输入

```
1 6 5
2 3 3 5 1 2 3
3 1 5 6 3 1 4
4 1 3 6 2 3 5
5 3 5 6 5 3 6
6 5 3 6 5 5 2
7 3 2 6 3 2 1
```

输出

```
1 5 4 3 3 2 1
2 5 4 3 2 2 1
3 4 4 4 3 2 1
4 3 3 3 3 2 1
5 2 2 2 2 2 1
6 1 1 1 1 1 1
```

【样例 3】

下发文件 `ex_anon_3.in`、`ex_anon_3.ans`。

【数据范围与提示】

- 对于所有测试数据，保证：
- $1 \leq n \leq 500, 1 \leq k \leq n, 1 \leq a_{i,j} \leq n$ 。

测试点编号	n 范围	特殊性质
1 ~ 3	≤ 50	无
4 ~ 5	≤ 500	$a_{i,j} \leq 2$
6 ~ 7	≤ 500	$k = 2$
8 ~ 10	≤ 500	无

Soyo 的定向 (soyo)

【题目描述】

Nagasaki Soyo 现在急着去帮 Anon 打扫花园.....

Soyo 面对了一棵具有 n 个结点、 $n - 1$ 条边的无向连通树。输入的第 i 条边的边权固定为 i 。

你需要帮 Soyo 为每一条边指定单向通行方向。

Soyo 不想上坡，因此她认定一个定向方案不合法当且仅当：图中存在一条包含至少两条边的路径，路径上的边权沿着定向行走方向构成上升序列。

求满足 Soyo 要求的合法定向方案总数量，答案对 998244353 取模。

【输入格式】

第一行一个整数 n 。

接下来 $n - 1$ 行，每行两个整数 x, y ，代表一条无向边；输入次序为第 $1, 2, \dots, n - 1$ 条边，第 i 条边权值为 i 。

【输出格式】

输出一行一个整数，合法定向方案数模 998244353。

【样例 1 输入】

```
1 4
2 1 2
3 2 3
4 3 4
```

【样例 1 输出】

```
1 5
```

【样例 2 输入】

```
1 4
2 2 3
3 1 2
4 3 4
```

【样例 2 输出】

14

【样例 3】

见下发文件 `ex_soyo_3.in/ans`。

【数据范围与提示】

对于所有测试数据，保证：

- $1 \leq n \leq 3 \times 10^5$ 。

测试点编号	n	特殊性质
1 ~ 2	15	无
3 ~ 5	3000	无
6	3×10^5	第 i 条边连接 $i, i + 1$
7	3×10^5	第 i 条边连接 n, i
8 ~ 10	3×10^5	无

Tomori 的石头 (tomori)

【题目描述】

Takamatsu Tomori 正偷偷在 Anon 的花园中捡石头.....

现有 n 颗石头在 Tomori 的桌子上排成一行，每颗大小均为 1 单位，第 i 颗的色深为 a_i 。

Tomori 要做出最好的石头。

她会执行 $n - 1$ 次合并操作：每次等概率选取当前相邻的两颗石头合并，然后丢弃一半。

具体地， a_i 与 a_{i+1} 合并后新色深为 $\frac{a_i+a_{i+1}}{2}$ ，总体积为 2，之后 Tomori 会抛弃一半，使得合并出的新石头大小仍为 1。

求最终仅剩一颗石头时，石头色深的数学期望，结果对模数 998244353 取模

【输入格式】

第一行一个正整数 n 。

接下来一行 n 个整数， a_i 代表第 i 颗石头的色深。

【输出格式】

一行一个整数， $E(x)$ 在模 998244353 意义下的值。

设最简分数 $\frac{a}{b}$ ，输出满足 $bx \equiv a \pmod{998244353}$ 且 $0 \leq x < 998244353$ 的唯一整数 x 。

【样例 1 输入】

```
1 3
2 11 45 14
```

【样例 1 输出】

```
1 374341653
```

【样例 2】

见下发文件 ex_tomori_2.in/ans。

【数据范围】

对于所有测试数据，保证：

- $1 \leq n \leq 10^6, 1 \leq a_i < 998244353$

测试点	n 上限
1	$n = 1$
2,3,4	$n = 2$
5,6	$n = 3$
7,8,9	≤ 10
10~11	≤ 200
12~15	≤ 2000
18~20	$\leq 10^6$

Taki 的融合 (taki)

【题目描述】

Shiina Taki 听说 Tomori 在合并石头，她也开始合并。但她的方法似乎与 Tomori 不太一样.....竟敢误石灯！

Taki 有 k 个石头序列，每个序列由若干个非负整数组成，记录着石头的重量。Taki 希望将**所有**序列中的石头**都**丢入面前的熔炉中。

熔炉上有一个容量计，会显示当前熔炉内所有石头的重量和。每次操作时，Taki 可以选择任意一个**非空序列**，将最靠前的**石头**丢入融合器中。**本次操作的代价**为这次操作**后**容量计显示的数值；**总代价**为所有操作的代价之和。

问题在于 Taki 可以通过选择不同序列的队头来改变石头的取出顺序。你需要帮助 Taki 求出**最小的总代价**。

然而，Tomori 说话很少，这导致 Taki 总是猜不透 Tomori 的心思.....因此石头序列会发生 q 次修改：每次选择一个已有的石头序列，在其**开头或末尾**加入一个新石头。你需要输出初始状态以及每次修改后的最小总代价。

形式化地，若当前共有 N 个元素，一次合法的取出方案，要求在操作序列中各序列元素内部保持原来的相对顺序。若最终操作序列为 $a_1, a_2, \dots, a_N$ ，则总代价为

$$\sum_{i=1}^N \sum_{j=1}^i a_j$$

多次修改，每次修改会在某个序列的**开头或末尾**添加一个数。你需要时刻维护并给出最小总代价。

【输入格式】

第一行包含两个正整数 k, q ，表示石头序列个数和修改次数。

接下来 k 行，每行开头一个整数 s_i ，表示当前序列的石头个数；接下来 s_i 个数，表示该序列的石头重量。

接下来 q 行，每行包含三个整数 op, t, v ：

- 若 $op = 0$ ，表示在序列 t 的**开头**加入一块重量为 v 的石头；
- 若 $op = 1$ ，表示在序列 t 的**末尾**加入一块重量为 v 的石头。

【输出格式】

输出共 $q + 1$ 行，每行一个整数：

- 第一行表示未修改时的最小代价；
- 接下来第 i 行表示第 i 次修改后的最小代价。

【样例 1】

输入

```
1 1 3
2 5 1 9 2 6 0
3 0 1 9
4 1 1 5
5 0 1 3
```

输出

```
1 59
2 113
3 145
4 169
```

样例 1 解释

样例 1 中只有一个石头序列，因此每次操作必须依次取出该序列的石头，取出顺序是确定的。

第一次修改前，答案为 $(1) + (1 + 9) + (1 + 9 + 2) + (1 + 9 + 2 + 6) + (1 + 9 + 2 + 6 + 0) = 59$

【样例 2】

输入

```
1 4 5
2 3 8 1 7
3 0
4 5 1 9 2 6 0
5 6 5 7 5 7 3 3
6 0 3 9
7 1 4 5
8 0 1 1
9 1 2 1
10 0 2 7
```

输出

```
1 453
2 588
3 664
```

4	681
5	699
6	826

【样例 3】

见下发文件 `ex_taki_3.in/ans`。

【数据范围与提示】

令 n 为所有序列总长度， v 为输入中权值的最大值。
对于所有测试点，满足 $n, q, k \leq 3 \times 10^5$ ， $v \leq 10^4$ 。
每个测试点的具体限制见下表：

测试点编号	$n \leq$	$q \leq$
1 ~ 3	500	500
4 ~ 6	5000	5000
7 ~ 12	无限制	0
13 ~ 15	无限制	20
16 ~ 20	无限制	无限制